

Assignment 06 – Windows, Tabs, and Iframes

1. Module Name

Module 1 – Automation with Selenium

2. Experiment Title

Windows, Tabs, and Iframes

3. Problem Statement

Create a Selenium Python automation script that demonstrates handling an iframe and multiple browser windows/tabs. The script should switch into an iframe, return to the main document, open a new window, switch between window handles, read the new window title, close the new window, and switch back to the original window.

4. Objective

The objective of this experiment is to understand how Selenium handles frames and multiple browser windows. The automation verifies practical use of `switch_to.frame()`, `default_content()`, `window_handles`, `switch_to.window()`, and `close()`.

5. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- VS Code
- WebDriverWait and Expected Conditions
- XPath and ID locators
- Iframes
- Browser windows/tabs
- Window handles
- Assertions

6. Source Code / Implementation Steps

1. Open the iframe practice page on The Internet.
2. Locate the iframe using its ID and switch into it with `switch_to.frame()`.
3. Read the text inside the iframe and verify that the frame was accessed successfully.
4. Return to the main document using `switch_to.default_content()`.
5. Open the browser windows practice page.
6. Store the original window handle using `current_window_handle`.
7. Click the 'Click Here' link to open a new window.
8. Retrieve all window handles using `window_handles` and switch to the new window.
9. Verify the new window title as 'New Window'.
10. Close the new window and switch back to the original window.
11. Verify that the original page is active and keep the browser open temporarily for screenshots.

7. Output / Labelled Screenshots

The following screenshots are captured during execution and stored in **Source_Code/Screenshots/**:

- Screenshot 1 – Iframe content displayed and accessed successfully
- Screenshot 2 – New window with its title
- Screenshot 3 – Original window after switching back
- Screenshot 4 – Successful execution message in the terminal

8. Result

The Selenium script successfully switched into the iframe, accessed its content, returned to the main page, opened and switched to a new browser window, retrieved its title, closed it, and returned to the original window. The required verification was completed successfully.

9. Observation

Selenium treats an iframe as a separate browsing context, so `switch_to.frame()` is required before interacting with elements inside it. For multiple windows or tabs, window handles are used to identify and switch between the available browser contexts. Closing the child window and explicitly switching back to the original handle ensures that automation continues in the correct window.

10. Conclusion

This experiment provided practical experience in handling iframes and multiple browser windows using Selenium Python. I learned how to switch between browsing contexts, retrieve window handles, verify a window title, close a child window, and return to the original window. The assignment was completed successfully.